

Clustering jerárquico¹

Una posible desventaja de K -means es que debemos especificar de antemano el número de clusters K .

El **clustering jerárquico** es una alternativa que:

- No requiere elegir inicialmente un número de clusters.
- Produce una representación gráfica en forma de árbol llamada **dendrograma**.

¹ An Introduction to Statistical Learning: with Applications in Python - Gareth James, Daniela Witten, Trevor Hastie, Robert Tibshirani, and Jonathan Taylor

Clustering jerárquico¹

Una posible desventaja de K -means es que debemos especificar de antemano el número de clusters K .

El **clustering jerárquico** es una alternativa que:

- No requiere elegir inicialmente un número de clusters.
- Produce una representación gráfica en forma de árbol llamada **dendrograma**.

Nos enfocaremos en el método **aglomerativo** (*bottom-up*):

Observaciones individuales $\rightarrow$ clusters pequeños $\rightarrow$ clusters grandes $\rightarrow$ un solo cluster.

¹ An Introduction to Statistical Learning: with Applications in Python - Gareth James, Daniela Witten, Trevor Hastie, Robert Tibshirani, and Jonathan Taylor

El dendrograma

Un **dendrograma** representa las observaciones y las fusiones sucesivas realizadas por el algoritmo.

- Cada **hoja** representa una observación.

El dendrograma

Un **dendrograma** representa las observaciones y las fusiones sucesivas realizadas por el algoritmo.

- Cada **hoja** representa una observación.
- Al subir en el árbol, las observaciones y clusters se van fusionando.

El dendrograma

Un **dendrograma** representa las observaciones y las fusiones sucesivas realizadas por el algoritmo.

- Cada **hoja** representa una observación.
- Al subir en el árbol, las observaciones y clusters se van fusionando.
- Fusiones a **menor altura** corresponden a grupos más similares.

Un **dendrograma** representa las observaciones y las fusiones sucesivas realizadas por el algoritmo.

- 10/110

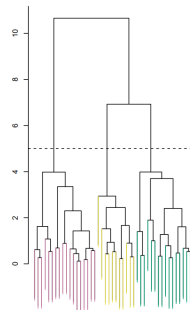
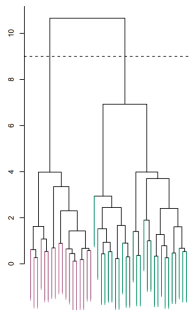
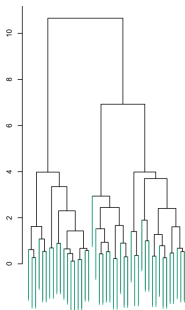
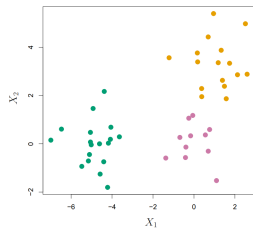
El dendrograma

Un **dendrograma** representa las observaciones y las fusiones sucesivas realizadas por el algoritmo.

- Cada **hoja** representa una observación.
- Al subir en el árbol, las observaciones y clusters se van fusionando.
- Fusiones a **menor altura** corresponden a grupos más similares.
- Fusiones a **mayor altura** corresponden a grupos más diferentes.

Para dos observaciones, la altura a la que sus ramas se fusionan por primera vez indica qué tan diferentes son.

Datos y dendrograma



Interpretación de un dendrograma

Importante

La proximidad horizontal entre dos observaciones en un dendrograma **no indica necesariamente que sean similares.**

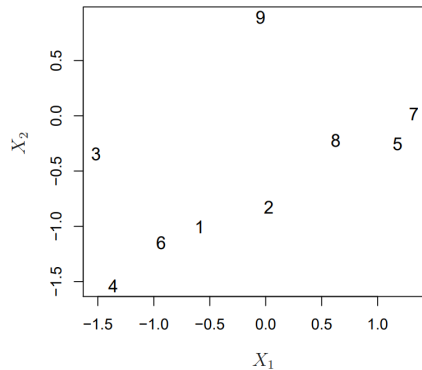
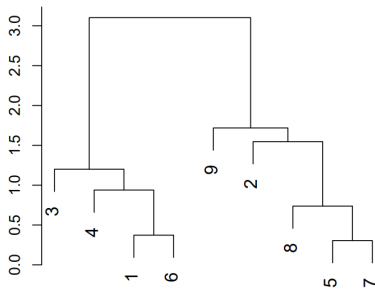
Interpretación de un dendrograma

Importante

La proximidad horizontal entre dos observaciones en un dendrograma **no indica necesariamente que sean similares**.

La similitud se determina observando la **altura de la primera fusión** de las ramas que contienen a las observaciones.

Ejemplo



¿Por qué no importa la posición horizontal?

En un dendrograma, las ramas pueden intercambiarse horizontalmente sin modificar la información representada.

¿Por qué no importa la posición horizontal?

En un dendrograma, las ramas pueden intercambiarse horizontalmente sin modificar la información representada.

Con n observaciones existen

$$2^{n-1}$$

posibles reordenamientos del dendrograma que representan exactamente la misma estructura jerárquica.

¿Por qué no importa la posición horizontal?

En un dendrograma, las ramas pueden intercambiarse horizontalmente sin modificar la información representada.

Con n observaciones existen

$$2^{n-1}$$

posibles reordenamientos del dendrograma que representan exactamente la misma estructura jerárquica.

Por lo tanto:

La información importante está en la altura de las fusiones.
--

Obtener clusters a partir del dendrograma

Para obtener clusters se realiza un **corte horizontal** en el dendrograma.

Obtener clusters a partir del dendrograma

Para obtener clusters se realiza un **corte horizontal** en el dendrograma.

Cada conjunto de observaciones conectado por debajo del corte corresponde a un cluster.

Obtener clusters a partir del dendrograma

Para obtener clusters se realiza un **corte horizontal** en el dendrograma.

Cada conjunto de observaciones conectado por debajo del corte corresponde a un cluster.

- Un corte alto $\Rightarrow$ pocos clusters.

Obtener clusters a partir del dendrograma

Para obtener clusters se realiza un **corte horizontal** en el dendrograma.

Cada conjunto de observaciones conectado por debajo del corte corresponde a un cluster.

- Un corte alto $\Rightarrow$ pocos clusters.
- Un corte bajo $\Rightarrow$ más clusters.

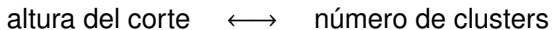
Obtener clusters a partir del dendrograma

Para obtener clusters se realiza un **corte horizontal** en el dendrograma.

Cada conjunto de observaciones conectado por debajo del corte corresponde a un cluster.

- Un corte alto $\Rightarrow$ pocos clusters.
- Un corte bajo $\Rightarrow$ más clusters.

La altura del corte juega un papel similar al parámetro K de K -means:



Una ventaja del clustering jerárquico

Un mismo dendrograma permite obtener diferentes números de clusters.

Una ventaja del clustering jerárquico

Un mismo dendrograma permite obtener diferentes números de clusters.

Por ejemplo, podemos cortar el mismo árbol para obtener:

$$K = 2, \quad K = 3, \quad K = 4, \quad \dots$$

Una ventaja del clustering jerárquico

Un mismo dendrograma permite obtener diferentes números de clusters.

Por ejemplo, podemos cortar el mismo árbol para obtener:

$$K = 2, \quad K = 3, \quad K = 4, \quad \dots$$

En la práctica, el dendrograma puede utilizarse para seleccionar visualmente un número razonable de clusters considerando:

- las alturas de las fusiones;
- separaciones grandes entre fusiones sucesivas;
- el número de grupos que resulte útil para la aplicación.

Estructura jerárquica

El término **jerárquico** significa que los clusters obtenidos mediante un corte deben estar contenidos dentro de los clusters obtenidos mediante un corte a mayor altura.

Estructura jerárquica

El término **jerárquico** significa que los clusters obtenidos mediante un corte deben estar contenidos dentro de los clusters obtenidos mediante un corte a mayor altura.

Es decir, las particiones están **anidadas**.

cluster grande $\supset$ clusters más pequeños $\supset$ observaciones.

Estructura jerárquica

El término **jerárquico** significa que los clusters obtenidos mediante un corte deben estar contenidos dentro de los clusters obtenidos mediante un corte a mayor altura.

Es decir, las particiones están **anidadas**.

cluster grande $\supset$ clusters más pequeños $\supset$ observaciones.

Esta suposición puede no ser apropiada para todos los conjuntos de datos.

Una limitación de la estructura jerárquica

Supongamos que tenemos personas:

- hombres y mujeres;
- estadounidenses, japoneses y franceses.

Una limitación de la estructura jerárquica

Supongamos que tenemos personas:

- hombres y mujeres;
- estadounidenses, japoneses y franceses.

La mejor partición en dos grupos podría ser:

{hombres}, {mujeres},

mientras que la mejor partición en tres grupos podría ser:

{estadounidenses}, {japoneses}, {franceses}.

Una limitación de la estructura jerárquica

Supongamos que tenemos personas:

- hombres y mujeres;
- estadounidenses, japoneses y franceses.

La mejor partición en dos grupos podría ser:

{hombres}, {mujeres},

mientras que la mejor partición en tres grupos podría ser:

{estadounidenses}, {japoneses}, {franceses}.

Estas particiones no están anidadas.

Consecuencia

En algunos problemas, clustering jerárquico puede producir una partición menos adecuada que *K*-means para un número fijo de clusters.

Algoritmo de clustering jerárquico

Supongamos que tenemos n observaciones.

1. Calcular las

$$\binom{n}{2} = \frac{n(n-1)}{2}$$

disimilitudes entre pares de observaciones.

Algoritmo de clustering jerárquico

Supongamos que tenemos n observaciones.

1. Calcular las

$$\binom{n}{2} = \frac{n(n-1)}{2}$$

disimilitudes entre pares de observaciones.

2. Inicialmente, considerar cada observación como un cluster:

$$C_1 = \{x_1\}, \dots, C_n = \{x_n\}.$$

Algoritmo de clustering jerárquico

Supongamos que tenemos n observaciones.

1. Calcular las

$$\binom{n}{2} = \frac{n(n-1)}{2}$$

disimilitudes entre pares de observaciones.

2. Inicialmente, considerar cada observación como un cluster:

$$C_1 = \{x_1\}, \dots, C_n = \{x_n\}.$$

3. Encontrar los dos clusters más similares y fusionarlos.

Algoritmo de clustering jerárquico

Supongamos que tenemos n observaciones.

1. Calcular las

$$\binom{n}{2} = \frac{n(n-1)}{2}$$

disimilitudes entre pares de observaciones.

2. Inicialmente, considerar cada observación como un cluster:

$$C_1 = \{x_1\}, \dots, C_n = \{x_n\}.$$

3. Encontrar los dos clusters más similares y fusionarlos.
4. Actualizar las disimilitudes entre los clusters restantes.

Algoritmo de clustering jerárquico

Supongamos que tenemos n observaciones.

1. Calcular las

$$\binom{n}{2} = \frac{n(n-1)}{2}$$

disimilitudes entre pares de observaciones.

2. Inicialmente, considerar cada observación como un cluster:

$$C_1 = \{x_1\}, \dots, C_n = \{x_n\}.$$

3. Encontrar los dos clusters más similares y fusionarlos.
4. Actualizar las disimilitudes entre los clusters restantes.
5. Repetir hasta que todas las observaciones pertenezcan a un único cluster.

Construcción del dendrograma

Inicialmente tenemos

n clusters.

Después de la primera fusión:

$n - 1$ clusters.

Después de la segunda:

$n - 2$ clusters.

⋮

Finalmente:

1 cluster.

Construcción del dendrograma

Inicialmente tenemos

n clusters.

Después de la primera fusión:

$n - 1$ clusters.

Después de la segunda:

$n - 2$ clusters.

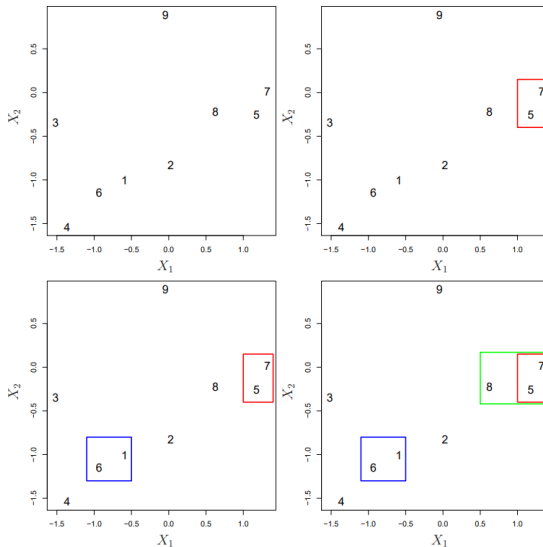
$\vdots$

Finalmente:

1 cluster.

La **altura** de cada fusión en el dendrograma viene dada por la disimilitud entre los dos clusters que se están fusionando.

Ejemplo



¿Cómo medimos la distancia entre clusters?

Para observaciones individuales podemos utilizar, por ejemplo, distancia Euclidiana:

$$d(x_i, x_j) = \sqrt{\sum_{k=1}^p (x_{ik} - x_{jk})^2}.$$

¿Cómo medimos la distancia entre clusters?

Para observaciones individuales podemos utilizar, por ejemplo, distancia Euclidiana:

$$d(x_i, x_j) = \sqrt{\sum_{k=1}^p (x_{ik} - x_{jk})^2}.$$

Pero después de fusionar observaciones surge una nueva pregunta:

¿Cómo definimos la disimilitud entre **dos clusters**?

¿Cómo medimos la distancia entre clusters?

Para observaciones individuales podemos utilizar, por ejemplo, distancia Euclidiana:

$$d(x_i, x_j) = \sqrt{\sum_{k=1}^p (x_{ik} - x_{jk})^2}.$$

Pero después de fusionar observaciones surge una nueva pregunta:

¿Cómo definimos la disimilitud entre **dos clusters**?

La respuesta depende del criterio de **linkage (o vinculación)** utilizado.

Linkage

Sean A y B dos clusters.

El **linkage** define cómo calcular una disimilitud

$$d(A, B)$$

a partir de las disimilitudes entre las observaciones que pertenecen a A y B .

Linkage

Sean A y B dos clusters.

El **linkage** define cómo calcular una disimilitud

$$d(A, B)$$

a partir de las disimilitudes entre las observaciones que pertenecen a A y B .

Los cuatro criterios más comunes son:

1. Complete linkage.
2. Single linkage.
3. Average linkage.
4. Centroid linkage.

Complete linkage

El **complete linkage** utiliza la mayor disimilitud entre observaciones pertenecientes a clusters diferentes:

$$d_{complete}(A, B) = \max_{\substack{i \in A \\ j \in B}} d(x_i, x_j)$$

Complete linkage

El **complete linkage** utiliza la mayor disimilitud entre observaciones pertenecientes a clusters diferentes:

$$d_{complete}(A, B) = \max_{\substack{i \in A \\ j \in B}} d(x_i, x_j)$$

Es decir:

1. Se calculan todas las distancias entre puntos de A y puntos de B .
2. Se toma la **mayor** de esas distancias.

Single linkage

El **single linkage** utiliza la menor disimilitud entre observaciones pertenecientes a clusters diferentes:

$$d_{single}(A, B) = \min_{\substack{i \in A \\ j \in B}} d(x_i, x_j)$$

Single linkage

El **single linkage** utiliza la menor disimilitud entre observaciones pertenecientes a clusters diferentes:

$$d_{single}(A, B) = \min_{\substack{i \in A \\ j \in B}} d(x_i, x_j)$$

Posible problema

Single linkage puede producir clusters largos o “encadenados”, en los que nuevas observaciones se agregan una por una a un cluster grande.

Average linkage

El **average linkage** utiliza el promedio de todas las disimilitudes entre observaciones de los dos clusters:

$$d_{average}(A, B) = \frac{1}{|A||B|} \sum_{i \in A} \sum_{j \in B} d(x_i, x_j)$$

Average linkage

El **average linkage** utiliza el promedio de todas las disimilitudes entre observaciones de los dos clusters:

$$d_{average}(A, B) = \frac{1}{|A||B|} \sum_{i \in A} \sum_{j \in B} d(x_i, x_j)$$

Por lo tanto, considera simultáneamente todas las relaciones entre los puntos de ambos clusters.

Centroid linkage

Sea

$$\bar{x}_A = \frac{1}{|A|} \sum_{i \in A} x_i$$

el centroide del cluster A , y de manera análoga $\bar{x}_B$ para B .

Centroid linkage

Sea

$$\bar{x}_A = \frac{1}{|A|} \sum_{i \in A} x_i$$

el centroide del cluster A , y de manera análoga $\bar{x}_B$ para B .

El **centroid linkage** define

$$d_{\text{centroid}}(A, B) = d(\bar{x}_A, \bar{x}_B)$$

Centroid linkage

Sea

$$\bar{x}_A = \frac{1}{|A|} \sum_{i \in A} x_i$$

el centroide del cluster A , y de manera análoga $\bar{x}_B$ para B .

El **centroid linkage** define

$$d_{\text{centroid}}(A, B) = d(\bar{x}_A, \bar{x}_B)$$

Es decir, compara los **centroides** de ambos clusters.

Desventaja

Puede generar **inversiones**: una nueva fusión puede aparecer a una altura menor que fusiones previas.

Comparación de criterios de linkage

Linkage	Disimilitud entre A y B
Complete	Mayor distancia entre dos observaciones de clusters diferentes.
Single	Menor distancia entre dos observaciones de clusters diferentes.
Average	Promedio de todas las distancias entre observaciones de clusters diferentes.
Centroid	Distancia entre los centroides de ambos clusters.

Comparación de criterios de linkage

Linkage	Disimilitud entre A y B
Complete	Mayor distancia entre dos observaciones de clusters diferentes.
Single	Menor distancia entre dos observaciones de clusters diferentes.
Average	Promedio de todas las distancias entre observaciones de clusters diferentes.
Centroid	Distancia entre los centroides de ambos clusters.

Average y complete linkage suelen generar clusters más **balanceados** que single linkage.

El linkage importa

El dendrograma obtenido puede cambiar considerablemente dependiendo del criterio de linkage.

El linkage importa

El dendrograma obtenido puede cambiar considerablemente dependiendo del criterio de linkage.

En general:

- **Complete linkage:** tiende a producir clusters compactos, evitando grupos con puntos muy alejados entre sí.
- **Average linkage:** utiliza la separación promedio entre los puntos de ambos clusters, por lo que es menos sensible a puntos extremos que complete linkage.
- **Single linkage:** puede producir el efecto de *chaining*.
- **Centroid linkage:** puede producir inversiones.

El linkage importa

El dendrograma obtenido puede cambiar considerablemente dependiendo del criterio de linkage.

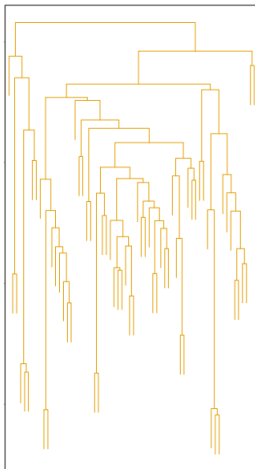
En general:

- **Complete linkage:** tiende a producir clusters compactos, evitando grupos con puntos muy alejados entre sí.
- **Average linkage:** utiliza la separación promedio entre los puntos de ambos clusters, por lo que es menos sensible a puntos extremos que complete linkage.
- **Single linkage:** puede producir el efecto de *chaining*.
- **Centroid linkage:** puede producir inversiones.

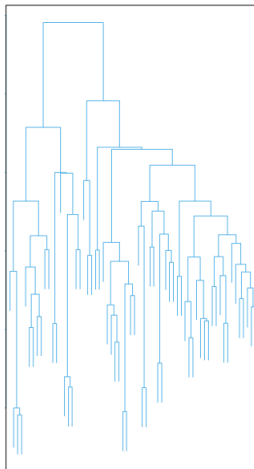
Por tanto, el linkage es una **decisión de modelado**, no simplemente un detalle computacional.

Tipos de linkage

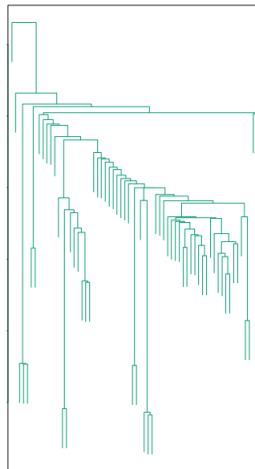
Average Linkage



Complete Linkage



Single Linkage



Elección de la medida de disimilitud

Hasta ahora hemos considerado principalmente la distancia Euclidiana:

$$d(x_i, x_j) = \sqrt{\sum_{k=1}^p (x_{ik} - x_{jk})^2}.$$

Elección de la medida de disimilitud

Hasta ahora hemos considerado principalmente la distancia Euclidiana:

$$d(x_i, x_j) = \sqrt{\sum_{k=1}^p (x_{ik} - x_{jk})^2}.$$

Sin embargo, existen otras opciones.

Por ejemplo, una distancia basada en **correlación** puede considerar similares a dos observaciones cuyos perfiles tienen una forma parecida, aunque sus magnitudes sean muy diferentes.

Elección de la medida de disimilitud

Hasta ahora hemos considerado principalmente la distancia Euclidiana:

$$d(x_i, x_j) = \sqrt{\sum_{k=1}^p (x_{ik} - x_{jk})^2}.$$

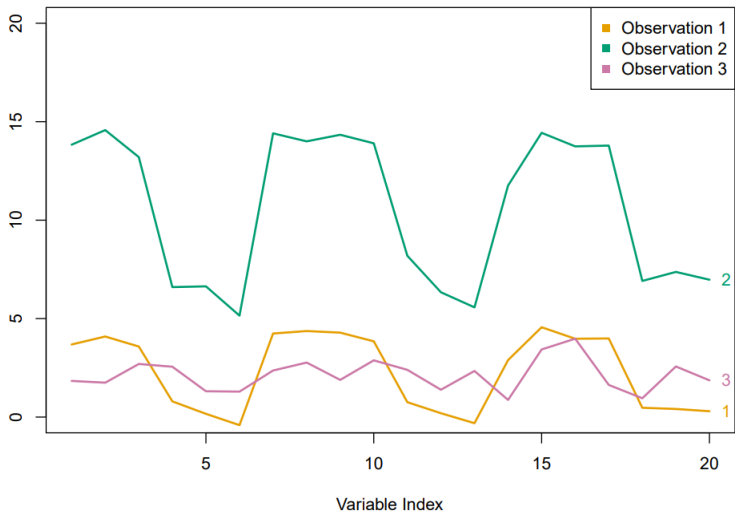
Sin embargo, existen otras opciones.

Por ejemplo, una distancia basada en **correlación** puede considerar similares a dos observaciones cuyos perfiles tienen una forma parecida, aunque sus magnitudes sean muy diferentes.

Idea

La distancia Euclidiana compara principalmente **magnitudes**; una medida basada en correlación puede comparar principalmente **patrones o formas**.

Ejemplo



Ejemplo: historial de compras

Supongamos una matriz donde

- cada fila representa un cliente;
- cada columna representa un producto;
- x_{ij} es el número de veces que el cliente i compró el producto j .

Ejemplo: historial de compras

Supongamos una matriz donde

- cada fila representa un cliente;
- cada columna representa un producto;
- x_{ij} es el número de veces que el cliente i compró el producto j .

Con distancia Euclidiana, dos clientes que compran pocos productos pueden resultar similares simplemente porque ambos tienen valores pequeños.

Con una distancia basada en correlación, dos clientes pueden considerarse similares si tienen **preferencias similares**, aunque uno compre mucho más que el otro.

Ejemplo: historial de compras

Supongamos una matriz donde

- cada fila representa un cliente;
- cada columna representa un producto;
- x_{ij} es el número de veces que el cliente i compró el producto j .

Con distancia Euclidiana, dos clientes que compren pocos productos pueden resultar similares simplemente porque ambos tienen valores pequeños.

Con una distancia basada en correlación, dos clientes pueden considerarse similares si tienen **preferencias similares**, aunque uno compre mucho más que el otro.

La medida de disimilitud debe elegirse según la aplicación.

¿Debemos estandarizar las variables?

La escala de las variables puede tener un efecto importante sobre las disimilitudes.

¿Debemos estandarizar las variables?

La escala de las variables puede tener un efecto importante sobre las disimilitudes.

Si una variable tiene una escala mucho mayor que otra, puede dominar la distancia Euclidiana.

¿Debemos estandarizar las variables?

La escala de las variables puede tener un efecto importante sobre las disimilitudes.

Si una variable tiene una escala mucho mayor que otra, puede dominar la distancia Euclidiana.

Una opción es estandarizar:

$$z_{ij} = \frac{x_{ij} - \bar{x}_j}{s_j},$$

de manera que cada variable tenga desviación estándar igual a uno.

¿Debemos estandarizar las variables?

La escala de las variables puede tener un efecto importante sobre las disimilitudes.

Si una variable tiene una escala mucho mayor que otra, puede dominar la distancia Euclidiana.

Una opción es estandarizar:

$$z_{ij} = \frac{x_{ij} - \bar{x}_j}{s_j},$$

de manera que cada variable tenga desviación estándar igual a uno.

Esto hace que las variables tengan una influencia más comparable sobre el clustering.

¿Siempre debemos estandarizar?

No necesariamente.

¿Siempre debemos estandarizar?

No necesariamente.

Estandarizar implica dar aproximadamente la misma importancia a cada variable en el cálculo de las disimilitudes.

¿Siempre debemos estandarizar?

No necesariamente.

Estandarizar implica dar aproximadamente la misma importancia a cada variable en el cálculo de las disimilitudes.

Esto puede ser conveniente cuando:

- las variables se encuentran en unidades diferentes;
- sus escalas son muy distintas;
- queremos que todas contribuyan de manera comparable.

¿Siempre debemos estandarizar?

No necesariamente.

Estandarizar implica dar aproximadamente la misma importancia a cada variable en el cálculo de las disimilitudes.

Esto puede ser conveniente cuando:

- las variables se encuentran en unidades diferentes;
- sus escalas son muy distintas;
- queremos que todas contribuyan de manera comparable.

Pero si una variable debe tener naturalmente mayor importancia, estandarizar podría no ser apropiado.

Conclusión

La decisión depende del contexto y del objetivo del análisis.

Clustering jerárquico: decisiones principales

Para aplicar clustering jerárquico debemos decidir:

1. ¿Cómo representar o escalar las variables?

Clustering jerárquico: decisiones principales

Para aplicar clustering jerárquico debemos decidir:

1. ¿Cómo representar o escalar las variables?
2. ¿Qué medida de disimilitud utilizar?

Clustering jerárquico: decisiones principales

Para aplicar clustering jerárquico debemos decidir:

1. ¿Cómo representar o escalar las variables?
2. ¿Qué medida de disimilitud utilizar?
3. ¿Qué criterio de linkage utilizar?

Clustering jerárquico: decisiones principales

Para aplicar clustering jerárquico debemos decidir:

1. ¿Cómo representar o escalar las variables?
2. ¿Qué medida de disimilitud utilizar?
3. ¿Qué criterio de linkage utilizar?
4. ¿Dónde cortar el dendrograma?

Clustering jerárquico: decisiones principales

Para aplicar clustering jerárquico debemos decidir:

1. ¿Cómo representar o escalar las variables?
2. ¿Qué medida de disimilitud utilizar?
3. ¿Qué criterio de linkage utilizar?
4. ¿Dónde cortar el dendrograma?

Todas estas decisiones pueden modificar significativamente los clusters obtenidos.

Clustering jerárquico en Python

Para implementar clustering jerárquico utilizamos `AgglomerativeClustering` de `sklearn.cluster`.

Para simplificar la notación definimos:

```
from sklearn.cluster import AgglomerativeClustering
HClust = AgglomerativeClustering
```

En este ejemplo utilizaremos estos datos

```
import numpy as np
import matplotlib.pyplot as plt
np.random.seed(0);
X = np.random.standard_normal((50,2));
X[:25,0] += 3;
X[:25,1] -= 4;

plt.scatter(X[:,0],X[:,1]);
```

y compararemos distintos tipos de *linkage* utilizando distancia Euclidiana.

Complete linkage

Primero realizamos clustering jerárquico utilizando **complete linkage**:

```
hc_comp = HClust(
    distance_threshold=0,
    n_clusters=None,
    linkage='complete'
)

hc_comp.fit(X)
```

- `n_clusters=None`: no fijamos un número de clusters.
- `distance_threshold=0`: construimos todo el dendrograma.
- `linkage='complete'`: usamos complete linkage.

Average y single linkage

Podemos cambiar fácilmente el tipo de *linkage*.

Average linkage:

```
hc_avg = HClust(
    distance_threshold=0,
    n_clusters=None,
    linkage='average'
)

hc_avg.fit(X)
```

Single linkage:

```
hc_sing = HClust(
    distance_threshold=0,
    n_clusters=None,
    linkage='single'
)

hc_sing.fit(X)
```

Utilizando una matriz de distancias

También podemos proporcionar directamente una **matriz de distancias previamente calculada**.

Primero construimos la matriz de distancias Euclidianas:

```
D = np.zeros((X.shape[0], X.shape[0]))
```

```
for i in range(X.shape[0]):
    x_ = np.multiply.outer(
        np.ones(X.shape[0]), X[i]
    )
    D[i] = np.sqrt(
        np.sum((X - x_)**2, 1)
    )
```

En este ejemplo,

$$D \in \mathbb{R}^{50 \times 50}.$$

Clustering con distancias precalculadas

Para indicar que proporcionaremos directamente las distancias, utilizamos

```
metric='precomputed'
```

Por ejemplo, con *single linkage*:

```
hc_sing_pre = HClust(
    distance_threshold=0,
    n_clusters=None,
    metric='precomputed',
    linkage='single'
)

hc_sing_pre.fit(D)
```

Ahora el algoritmo utiliza directamente la matriz de disimilitudes D .

Construcción del dendrograma

Para graficar el dendrograma utilizamos:

```
from scipy.cluster.hierarchy import dendrogram
```

Sin embargo, `dendrogram()` necesita una representación conocida como **linkage matrix**.

AgglomerativeClustering no produce directamente esta matriz, por lo que se puede utilizar:

```
# !pip install ISLP
from ISLP.cluster import compute_linkage

linkage_comp = compute_linkage(hc_comp)
```


Graficar el dendrograma

Eliminamos inicialmente el coloreado automático de las ramas:

```
cargs = {
    'color_threshold': -np.inf,
    'above_threshold_color': 'black'
}
```

Después graficamos:

```
dendrogram(
    linkage_comp,
    **cargs
)
```

Los números en la parte inferior del dendrograma identifican las observaciones.

Cortar visualmente el dendrograma

Podemos utilizar `color_threshold` para visualizar un corte del dendrograma.

Por ejemplo, para un corte a altura 4:

```
dendrogram(
    linkage_comp,
    color_threshold=4,
    above_threshold_color='black'
);
```

Las ramas que se fusionan por encima de la altura 4 se muestran en negro.

Obtener los clusters del dendrograma

Para obtener las etiquetas de los clusters correspondientes a un corte utilizamos

```
from scipy.cluster.hierarchy import cut_tree
```

Por ejemplo, si queremos obtener exactamente 4 clusters:

```
cut_tree(
    linkage_comp,
    n_clusters=4
).T
```

El resultado asigna a cada observación una etiqueta:

0, 1, 2, 3.

Cortar por número de clusters

También hubiéramos podido especificar directamente el número de clusters en

```
HClust(n_clusters=4, ...)
```

Sin embargo, existe una diferencia importante:

- Con `cut_tree()`, podemos utilizar el mismo dendrograma y realizar distintos cortes.
- Si cambiamos `n_clusters` en `AgglomerativeClustering`, debemos volver a calcular el clustering para cada corte.

Cortar por altura

En lugar de especificar el número de clusters, también podemos cortar el dendrograma a una **altura determinada**.

Por ejemplo:

```
cut_tree(
  linkage_comp,
  height=5
).T
```

Esto devuelve los clusters existentes al cortar el dendrograma a la altura

$$h = 5.$$

Graficar los clusters en los datos

```
labels = cut_tree(  
    linkage_comp,  
    n_clusters=4  
) .reshape(-1)  
  
plt.scatter(  
    X[:, 0],  
    X[:, 1],  
    c=labels  
)
```

Estandarizar antes del clustering

También podemos estandarizar las variables antes de realizar clustering jerárquico.

Utilizamos:

```
from sklearn.preprocessing import StandardScaler

scaler = StandardScaler()
X_scale = scaler.fit_transform(X)
```

Después realizamos nuevamente el clustering:

```
hc_comp_scale = HClust(
    distance_threshold=0,
    n_clusters=None,
    linkage='complete'
).fit(X_scale)
```

Dendrograma con variables estandarizadas

Construimos la matriz de linkage:

```
linkage_comp_scale = compute_linkage(
    hc_comp_scale
)
```

y graficamos:

```
fig, ax = plt.subplots(1, 1, figsize=(8, 8))

dendrogram(
    linkage_comp_scale,
    ax=ax,
    **cargs
)

ax.set_title(
    "Hierarchical Clustering with Scaled Features"
)
```


Comparación de dendrogramas

```
fig, ax = plt.subplots(1, 2, figsize=(12, 6))

# Datos escalados
dendrogram(linkage_comp_scale, ax=ax[0], **cargs)
ax[0].set_title("Hierarchical Clustering with Scaled Features")

# Datos originales
dendrogram(linkage_comp, ax=ax[1], **cargs)
ax[1].set_title("Hierarchical Clustering with Original Features")

plt.tight_layout()
```

Comparación de clusters

```
n_clusters = 4

labels_scale = cut_tree(linkage_comp_scale,n_clusters=n_clusters).reshape(-1)
labels = cut_tree(linkage_comp,n_clusters=n_clusters).reshape(-1)

fig, ax = plt.subplots(1, 2, figsize=(12, 5))

# Clusters obtenidos usando datos escalados
ax[0].scatter( X[:, 0],X[:, 1],c=labels_scale,cmap='tab10')
ax[0].set_title("Clusters from Scaled Features")
ax[0].set_xlabel("X1")
ax[0].set_ylabel("X2")

# Clusters obtenidos usando datos originales
ax[1].scatter(X[:, 0],X[:, 1],c=labels,cmap='tab10')
ax[1].set_title("Clusters from Original Features")
ax[1].set_xlabel("X1")
ax[1].set_ylabel("X2")

plt.tight_layout()
```


Disimilitud basada en correlación

También podemos utilizar la **correlación** entre observaciones para definir una medida de disimilitud.

La correlación mide la similitud entre los valores de las características de dos observaciones.

Disimilitud basada en correlación

También podemos utilizar la **correlación** entre observaciones para definir una medida de disimilitud.

La correlación mide la similitud entre los valores de las características de dos observaciones.

A partir de la matriz de correlación podemos definir:

$$D_{ij} = 1 - \text{corr}(x_i, x_j)$$

De esta forma:

- correlación cercana a 1 $\Rightarrow$ disimilitud cercana a 0;
- correlación cercana a 0 $\Rightarrow$ disimilitud cercana a 1;
- correlación cercana a $-1 \Rightarrow$ disimilitud cercana a 2.

Clustering con distancia basada en correlación

Primero generamos los datos:

```
np.random.seed(42)

n = 30

C1 = np.array([ 2, 0, -2])
C2 = np.array([-2, 2, 0])
C3 = np.array([ 0, -2, 2])

X1 = C1 + 0.25 * np.random.standard_normal((n, 3))
X2 = C2 + 0.25 * np.random.standard_normal((n, 3))
X3 = C3 + 0.25 * np.random.standard_normal((n, 3))

X = np.vstack((X1, X2, X3))
```

Clustering con distancia basada en correlación

Luego calculamos la matriz de disimilitud:

```
corD = 1 - np.corrcoef(X)
```

Por tanto,

$$\text{corD}_{ij} = 1 - \text{corr}(x_i, x_j).$$

Complete linkage con correlación

Como ya tenemos calculada la matriz de disimilitudes, utilizamos:

```
hc_cor = HClust(  
    linkage='complete',  
    distance_threshold=0,  
    n_clusters=None,  
    metric='precomputed'  
)  
  
hc_cor.fit(corD)
```

Complete linkage con correlación

Finalmente construimos y graficamos el dendrograma:

```
linkage_cor = compute_linkage(hc_cor)

fig, ax = plt.subplots(1, 1, figsize=(8, 8))

dendrogram(linkage_cor, ax=ax, **cargs)

ax.set_title(
    "Complete Linkage with "
    "Correlation-Based Dissimilarity"
)
```

Graficar ejemplo 3D

```
labels = cut_tree(  
    linkage_cor,  
    n_clusters=4  
) .reshape(-1)  
  
fig = plt.figure(figsize=(8, 8))  
ax = fig.add_subplot(111, projection='3d')  
  
ax.scatter(  
    X[:, 0],  
    X[:, 1],  
    X[:, 2],  
    c=labels  
)  
  
ax.set_xlabel('X1')  
ax.set_ylabel('X2')  
ax.set_zlabel('X3')  
ax.set_title('Hierarchical Clustering with Correlation-Based  
Dissimilarity')
```

¡Gracias!